ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 4

4ИСИП-1023 Жукова Маргарита

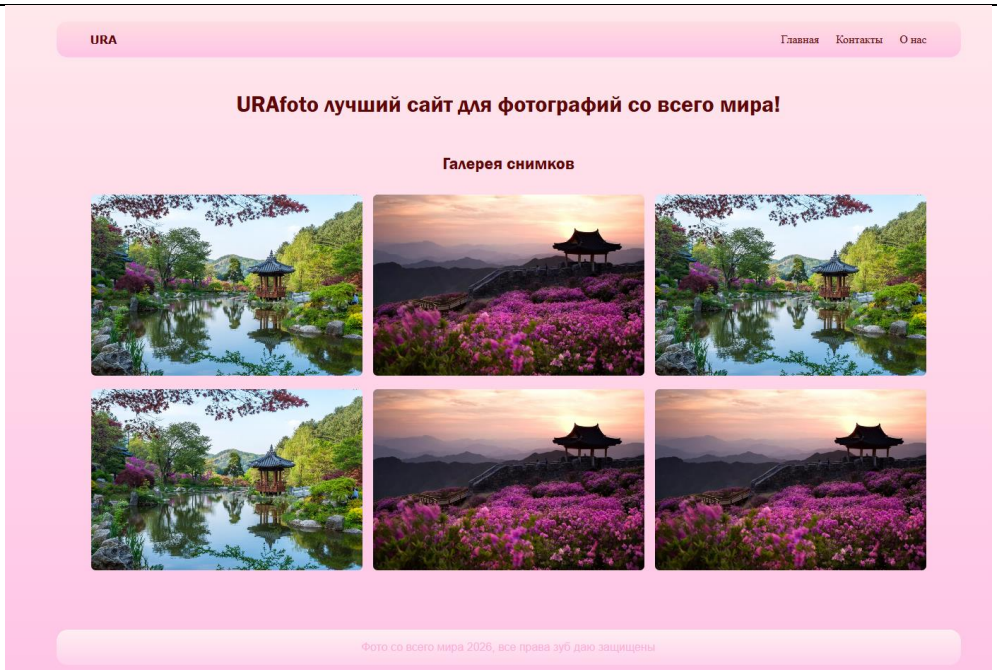
Цель работы – исследование методов оптимизации для мобильных устройств, изучение медиа-запросов (Media Queries) CSS и создание полноценного отзывчивого/адаптивного веб-дизайна для многостраничного веб-сайта.

Выполнение практических шагов

Шаг 1: Создание макета с адаптивными стилями

Был разработан многостраничный веб-сайт фотогалереи «URA» (страницы index.html, about.html, contacts.html). Для обеспечения корректной работы адаптивности на всех мобильных устройствах в тег <head> каждого документа включен метатег viewport:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



Шаг 2: Медиа-запросы для адаптивного дизайна

В файле style.css прописаны каскадные медиа-запросы с использованием контрольных точек (breakpoints) под экраны различных типов. Использована сетка Flexbox с перестраиваемой шириной карточек gallery-блоков (.card) и переключением флекс-направления (flex-direction: column) для шапки и футера.

```
/* Базовые стили сеток и карточек */
.content {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 15px;
    flex-wrap: wrap;
    margin: 0 auto;
    flex: 1;
}

.card {
    width: 30%;
}

.card img {
    width: 100%;
    height: 16rem;
    object-fit: cover;
    border-radius: 0.4rem;
}

/* 1. Мониторы (от 1200px / 75rem) */
@media (min-width: 75rem) {
    body {
        max-width: 80rem;
        margin-left: auto;
        margin-right: auto;
        padding: 0 3rem;
    }
    h1, p, h2 {
        flex-direction: column;
        gap: 1rem;
        text-align: center;
    }
}

/* 2. Ноутбуки (до 1200px / 75rem) */
@media (max-width: 75rem) {
    .card {
        width: 30%;
    }
    .card img {
        height: 14rem;
    }
    h1, p, h2 {
        flex-direction: column;
        gap: 1rem;
        text-align: center;
```

```
        }
    }

    /* 3. Планшеты (до 960px / 60rem) */
    @media (max-width: 60rem) {
        .card {
            width: 45%;
        }
        .card img {
            height: 15rem;
        }
        h1, p, h2 {
            flex-direction: column;
            gap: 1rem;
            text-align: center;
        }
    }

    /* 4. Мобильные устройства (до 640px / 40rem) */
    @media (max-width: 40rem) {
        .header {
            flex-direction: column;
            gap: 1rem;
            text-align: center;
        }
        .card {
            width: 100%;
        }
        .card img {
            height: 16rem;
        }
        .footer {
            flex-direction: column;
            gap: 1rem;
            text-align: center;
        }
        h1, p, h2 {
            flex-direction: column;
            gap: 1rem;
            text-align: center;
        }
    }
```

URA

Главная Контакты О нас

URAfoto лучший сайт для фотографий со всего мира!

Галерея снимков






URA

Главная Контакты О нас

URAfoto лучший сайт для фотографий со всего мира!

Галерея снимков

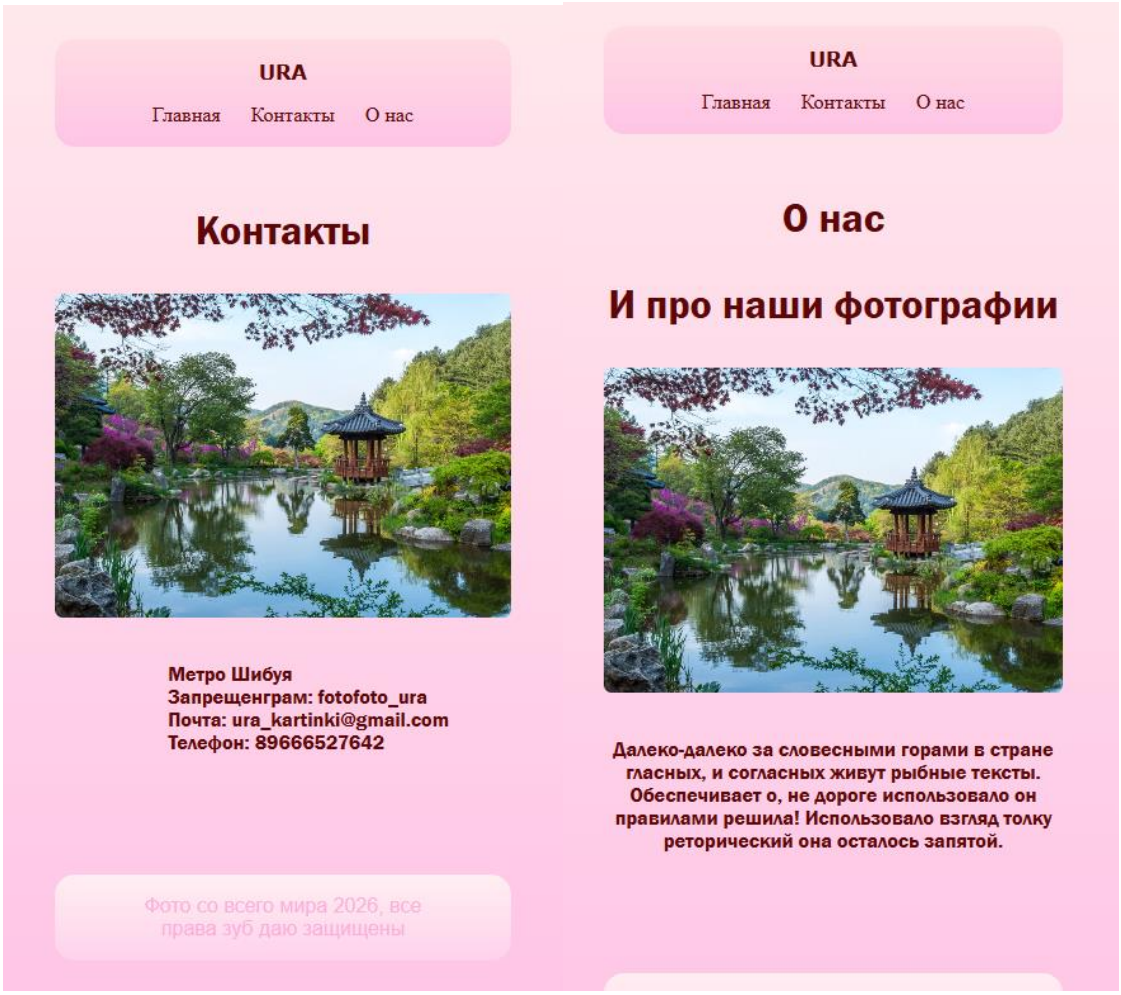



Шаг 3: Оптимизация изображений для мобильных устройств

В рамках работы оптимизировано отображение медиа-контента. Для сохранения пропорций и предотвращения искажения фотографий при изменении ширины экрана использовано CSS-свойство object-fit: cover с адаптивной высотой карточек. Это гарантирует аккуратную подгонку контента под дисплеи с любой плотностью пикселей (включая Retina).

В ходе выполнения самостоятельной работы создание адаптивного дизайна было продолжено и полностью распространено на остальные страницы веб-приложения – «О нас» (about.html) и «Контакты» (contacts.html). Все страницы сайта имеют единую гибкую структуру элементов на базе Flexbox и единый подключаемый stylesheet (style.css). Текстовые блоки, описания, списки контактов и интерактивная карта оптимизированы под мобильные экраны: при изменении ширины окна элементы шапки (шапка сайта .header и навигация .navbar) перестраиваются из горизонтальной строки в вертикальный стек с увеличенными областями нажатия (touch targets), предотвращая случайные клики.

Все страницы и элементы макета прошли полное тестирование на отзывчивость в эмуляторе браузера Chrome DevTools в широком диапазоне разрешений (от 320px до 1920px), а также на реальных мобильных устройствах. Проверка подтвердила отсутствие горизонтальной полосы прокрутки (overflow-x), корректное масштабирование текстовых блоков и сохранность визуальной иерархии сайта.



Ответы на контрольные вопросы

1. В чем принципиальная разница между «адаптивным дизайном» (adaptive) и «отзывчивым дизайном» (responsive) веб-страниц, и какой из подходов реализуется с помощью медиа-запросов в данной работе?

– Отзывчивый (Responsive) дизайн – макет является полностью «резиновым» и плавно подстраивается под любой размер экрана в реальном времени благодаря относительным единицам (%, vw, rem, flex/grid).

– Адаптивный (Adaptive) дизайн – верстка подстраивается под жестко заданные точки останова (breakpoints) и переключается между фиксированными макетами.

В данной работе реализуется смешанный подход с преобладанием адаптивного дизайна, так как в style.css прописаны четкие медиа-запросы (@media) на ключевые брейкпоинты (75rem/1200px, 60rem/960px, 40rem/640px), переключающие сетку карточек и структуру шапки.

2. Объясните, как браузер интерпретирует медиа-запросы. Что произойдет, если для устройства с шириной экрана 750px будут одновременно активны два медиа-запроса: (max-width: 600px) и (min-width: 768px) и (max-width: 959px)?

Браузер считывает параметры устройства/окна и вычисляет истинность (true/false) каждого условия медиа-запроса. Если условие true, браузер применяет блоки стилей внутри него. При совпадении нескольких условий действует каскад CSS (более специфичные или расположенные ниже в файле правила перекрывают предыдущие).

При ширине экрана 750px:

Запрос (max-width: 600px) вернет FALSE (750 > 600).

Запрос (min-width: 768px) and (max-width: 959px) вернет FALSE, так как условие min-width: 768px не выполнено (750 < 768).

Итог: Ни один из этих двух медиа-запросов не сработает. Браузер отобразит элемент по его базовым стилям.

3. В работе предложены жесткие значения ширины экрана (600px, 768px, 1280px). Почему на практике все чаще используют подход «Mobile First» с min-width, а не «Desktop First» с max-width?

Подход Mobile First заставляет сначала проектировать верстку под узкие мобильные экраны (базовый CSS), а затем по мере роста экрана дописывать стили через min-width.

Причины преобладания Mobile First:

– Производительность: Мобильные устройства со слабым процессором загружают чистый минималистичный код без необходимости отменять или переопределять тяжелые десктопные стили.

– Статистика: Более 60% мирового веб-трафика приходится на мобильные устройства.

– Чистота кода: Добавлять элементы по мере расширения экрана стилистически проще, чем скрывать и переписывать десктопную верстку под смартфоны.

4. Проанализируйте приведенный HTML-код с блоками .qw. Как медиа-запросы влияют на визуальное представление блоков при изменении размера экрана?

Медиа-запросы динамически меняют геометрию, размеры и расположение элементов в потоке документа. В исследуемом коде при изменении ширины экрана медиа-запросы пересчитывают процентную ширину блоков (переходя от 30% на десктопе к 45% на планшете и 100% на смартфоне), изменяют флекс-направление конвейера (.header и .footer переходят в column) и выравнивание текста, предотвращая появление горизонтального скролла.

5. В примере с блоками используется свойство order для изменения их очередности. Приведите пример реального интерфейса, где изменение порядка блоков на мобильных устройствах критически важно для удобства пользователя.

Классический пример – Карточка товара в интернет-магазине. На десктопе слева расположен крупный блок с галереей фотографий, а справа — цена, кнопка «Купить» и характеристики. На мобильном устройстве без свойства order галерея заняла бы несколько экранов прокрутки вниз, из-за чего кнопка «Купить» ушла бы далеко за пределы первого экрана. С помощью flex order: -1 кнопку покупки и цену на смартфоне поднимают в самый верх над галереей, улучшая UX и конверсию.

6. В коде блоки скрываются с помощью display: none. Почему скрытие элементов на мобильных устройствах может не дать существенного выигрыша в производительности загрузки, если не применять дополнительные техники?

Свойство display: none исключает элемент из дерева рендеринга (Render Tree) и визуального отображения, но НЕ отменяет загрузку ресурсов. Браузер все равно выполнит HTTP-запросы и скачает тяжелые картинки, видео или скрипты, прописанные в HTML. Для реальной оптимизации загрузки необходимо использовать атрибут loading="lazy", динамическую загрузку через JS или элемент <picture>.

7. Какую технику из первой лабораторной работы (или ее развитие) следует применить к изображениям, чтобы они корректно и быстро загружались на мобильных устройствах с разной плотностью пикселей (Retina-экраны)?

Следует применять адаптивные изображения с использованием тега <img> с атрибутами srcset и sizes, либо контейнер <picture> с тегами <source media="...">. Это позволяет браузеру автоматически выбирать и скачивать файл нужного разрешения в зависимости от плотности пикселей экрана (1x, 2x Retina, 3x) и текущей ширины viewport.

8. В шаге 3 упоминаются иконочные шрифты. В чем преимущество использования иконочных шрифтов (или SVG-иконок) перед использованием

растровых изображений (PNG) для маленьких элементов интерфейса с точки зрения адаптивности и производительности?

Векторная масштабируемость: Иконочные шрифты и SVG не теряют чёткости при любом масштабировании и на любых Retina-дисплеях.

Малый размер и оптимизация: Вектор занимает в разы меньше килобайт, чем аналогичный набор PNG в нескольких разрешениях.

Гибкость стилизации: Цвет, размер и тени векторных иконок легко меняются через стандартный CSS (color, font-size, fill), без необходимости перерисовывать картинки в графическом редакторе.

9. Для работы примера используется тег <meta name="viewport" content="width=device-width, initial-scale=1.0">. Что произойдет с отображением адаптивного макета, если удалить этот тег? Объясните физический смысл его атрибутов.

Если удалить этот тег, мобильный браузер включит режим эмуляции десктопного экрана (шириной по умолчанию 980px) и искусственно уменьшит всю страницу под размер дисплея телефона. Сайт станет микроскопическим, а медиа-запросы перестанут корректно срабатывать.

Физический смысл атрибутов:

– width=device-width – приравнивает ширину области просмотра (layout viewport) к физической ширине экрана устройства в независимых пикселях (CSS pixels).

– initial-scale=1.0 – устанавливает начальный коэффициент масштабирования страницы 1:1 при загрузке.

10. В работе предлагается тестировать на реальных устройствах. Назовите два способа эмуляции мобильных устройств, доступных в браузерах (например, Chrome DevTools), и укажите, в чем главное ограничение такого эмулятора по сравнению с реальным физическим устройством.

Два способа эмуляции в Chrome DevTools:

– Режим Device Mode (сочетание клавиш Ctrl + Shift + M / Cmd + Option + M).

– Использование встроенных пресетов конкретных устройств (iPhone, Pixel, iPad) с настройкой throttling (искусственного замедления сети и CPU).

Главное ограничение эмулятора: Эмулятор имитирует лишь размеры viewport и User-Agent, но не воспроизводит физический мобильный графический движок, реальные задержки сенсорных тач-событий (touch events) и аппаратные ограничения мобильного процессора при рендеринге сложных стилей и JS.

Вывод: В ходе выполнения практической работы были успешно изучены и применены на практике ключевые принципы мобильной оптимизации и создания адаптивного веб-дизайна. На примере веб-сайта фотогалереи «URA» освоена работа с метатегом viewport, принципами отзывчивой сетки Flexbox и каскадными медиа-запросами CSS (@media). Все страницы приложения («Главная», «О нас», «Контакты») были полностью адаптированы и проверены в эмуляторе Chrome DevTools на различных точках останова. Проведенный анализ контрольных вопросов позволил углубить понимание разницы между Responsive и Adaptive подходами, принципов Mobile First, особенностей работы браузерного каскада и методов оптимизации медиа-контента для современных экранов высочайшей плотности.